Cpp 数据类型
short int long long long
char（字符） bool

注释
//单行
/*多行注释*/

Sizeof 关键字（求占用大小）
语法：
Sizeof（数据类型/变量）
例如：
short num1 =10；
Cout<<" "<<sizeof(num1)<<endl;

转义字符
\n 换行
\\ 反斜杠
\t 水平视角

变量：数据类型+标识符
int a
a = 数字
int a = 数字（初始化）
bool b = true
（布尔里面等于数字（非零）也相当于 true，其他为 false）
float c = 3.14f
double d = 1.43854648
char e = '字符串' （用单引号）
cout≪A≪"/t"≪b≪c≪endl；
（t 表示 table，输出为：a b，即自动空格）
结束都用分号
const 数据类型 量 = 数字（定义常量

&& 相当于：与
ll 相当于：或
!: 非

&按位与运算
A=101000101
B=000101010
&运算为：010010000（相同为 1，不同为 0）

|按位或运算
A=101000101
B=000101010
|运算为：101101111（同个位置只要有 1 就算 1，无则为 0）

^
A=101000101
B=000101010
同个位置有不同则为 1，相同为 0

~
换位符号
A=101000101
B=000101010
则~A=010111010(所有 1 换成 0，所有 0 换成 1)

<<
左移符号，“a<<2”即整个数乘以 4，最后面补上两个数字 0
>>
右移符号
A>>2,则左边补 0，右边舍弃
C++中，if 语句格式
If （条件）
{
内容；
}

也可以为：
If(真假)
{
代码；
}
Else；
{
代码
}

如果只有一条语句执行，那么大括号可以省略
If 语句的表达式必须是布尔类型
使用逻辑运算符时应该使用括号确定运算的优先级别，避免出现逻辑错误

If 语句中不要使用 0<a<200
不然回先判断 0<a 成立为 true，即 1，1<200 成立，那么 a 等于 6000 也成立

TIP：不要出现连等，必须用与（&&）or 或（||）

If 可以嵌套

还有 if（）
{
代码；
}
elif()
{
代码；
}
elif()
{
代码；
}
……

Switch 语句
格式：
switch（值）
{
    case 常量 1：
        代码 1；
        break；
    case 常量 2：
        代码 2；
        break；
    ……
    default：
        代码 3
}

Switch 语句中常量必须唯一且不重复
Switch 语句中 default 分支可写可不写
每个 case 分支必须使用 break
可嵌套
如果分支语句中的代码太多，会报错。 此时可以用大括号括起来，再在外面写 break

For 语句
格式：for（代码 1；代码 2；代码 3）
{
需要重复执行的代码；

}

代码1、2、3都可以省略，但需要保留分号
代码2一般是个范围

While语句
格式：while（条件表达式）{
循环体语句
}

While语句中的变量需要被初始化

Do-while语句
Do{
循环语句}
While（）；

跳转语句
三种：break（跳出循环），continue（结束当前循环，进入下一循环），goto（用于跳转到标志语句）（标志语句必须在goto之前被定义）

引用
引用定义只能在声明中进行初始化，只能引用被初始化的值
定义：
变量名的类型标识符　　&引用名=变量名

对引用的操作和对引用变量的操作是完全等价的
引用的类型的必须和对应的变量类型相同

数组

Type 数组名字【数组长度，必须是整数】
E. g.:　int arr[10]={1，2，3，4，5，6，7，8，9，10}；
　　访问：通过下标访问
例如：arr[0];//1
下标都是从0开始访问不是1，如果访问的数字超出数容量，则显示0

数据类型 数组名字【数据长度】={值1，值2，值3……}
数据类型 数组名【】={值1，值2……}；
数组类型 数组名字【】={值1，值2……}：

特点：放在一块连续的内存空间，数组中每个元素都是相同类型

数组名的作用：
统计长度 sizeof（名）
名可以是数组名，也可以是 sizeof(arr[0])

可以通过数组名查看数组首地址
直接 cout <<”数组的首地址为：”<<arr<<endl;
一般输出十六进制（可以在前面加一个 int 表示查看整数类型）

查看元素的话：（int）&arr[下标]
元素与元素之间相差四个字节
数组名字等于常量，不可以赋值

函数
返回类型（int……没有返回值用 void）+函数名字（参数类型 1+参数名 1，参数类型 2+参数名字 2）
{
//函数体
}
（参数可有可无）

做一个假发有返回值的函数

```
 Int add(int x,int y)
{
    return x+y;
}
Int main()
{
Int a=add(2,3);
Cout<<a<<endl;
}
```

再比如：

```
void f()
{
Cout <<”你好”<<endl;
}
```

返回值和期望值一样
先声明再调用

命名空间：可以包含变量、函数、和其他命名空间

定义方法：

```
namespace namespace_name{
    Int varl:
    Void funcl();
    Namespace sub_namespace{
        //sub-namespace content
    }
}
```

例如：

```
namespace YIHU
```

```
{
    Int age =18;
    Void fun()
     {
          Cout <<" age" <<age<<endl;
     }
     Namespace sUB_YIHU
     {
          Void fun()
          {
               Cout <<" sub_fun 函数" <<endl;
          }
     }
}

Int main()
{
YIHU::fun;
//意思为调用 name……命名空间内的函数 fun()(但是引用的是第一层)

YIHU:: sUB_YIHU::fun;
//意思为调用下面新的命名空间里面的函数
}
```

注意
1、C++中标识符是唯一的，如果有两个标识符会报错，命名空间是一种将标识符分组的机制
2、 可以创建多个命名空间，避免同名标识符之间冲突
3、 在使用标识符的过程中，可以使用:: 指明属于哪个命名空间
4、 没有指定命名空间则会指定使用全局命名空间（using namespace std;）
(
指的就是： int fun()
Int(引用)
)

类

通过分装数据和代码组织为独立整体来处理
定义方式：

```
class 类名{
//访问属性
//类中成员
};
```

举例子：

```
class student
{
    Char name [20];
    Int age;
    Int id;

    Void  speak()
    {
        Cout <<" 今年我 "<<age<<" 岁 "<<endl;
    }
    Void playball()
    {
        Speak();
        Cout<< …… 
    }
}
```

（这个是定义模板，后面可以用来输入一个数值）

后面需要用到的时候就：

```
int main  ( )
{
        Student stu;
        //前面的 student 是模板，后面的 stu 是新定义的模板
)
```

类中成员：变量+函数

访问属性：

public ：共有成员可以在类内外直接访问

Protected ：保护成员，类中可以访问，类外不能访问

Private ：私有成员，类中可以访问，类外不能直接访问

```
Class a
{
    Private:
        int a;
        Int b;
    Public:
        Int c;
}
Int main()
{
```

}

String copy===字符串拷贝，简称为：strcpy
Char []不可以直接复制，string 可以直接赋值

可能会：
Void setAge (int x)
{
Age =x;
//表示设置 age 的值
}
Int getAge()
{
    Return age;
    //读取 age 的值，并返回给 age
}

保护和私有在类中可以访问，但是在类外不能访问，所以需要定义一个共有的函数版主访问

To string：表示把数字转化为字符串

Get 读取，set 修改

私有量不能访问时候，提供两个变量：
setter（设置器）
通常没有返回值，用来给私有变量赋值，有参数，负责把外部传进来的值赋给内部变量

Getter（取值器）
用来读取私有变量，但有返回值，负责把内部值返回给外部

面向对象

将数据和操作数据打包
1、类；
用 case 关键字来定义
2、对象（被打包后的数据）；
3、继承
从副类中继承新的类
4、多态
多个方法
5、封装
Public protect private

编程原则

SOLID:
单一职责：一个类的工作必须单一
开放封闭：增加功能按时不能该代码
里氏替换：子类可以替换父类使用
接口隔离：模块通过多个接口交互
Be like:一个人交流所有部门（不可）
要多个人对应多个部门
依赖倒置：
传统：高层依赖低层
原则指的是高层依赖抽象层而不是依赖底层

DRY 原则
不要重复代码
尽可能通过抽象和封装来复用代码

KISS 保持代码简洁、易读

YAGNI 原则
只实现目前需要的功能，不优化

高内聚，低耦合
一个模块只负责完成一个单一的功能，低耦合指不同模块和类之间的相互一类成都越低越好
减少互相帮忙，依赖低

数据的逻辑结构

线性结构（只有一个开始和一个终端节点）
非线性结构（分支）

数据的树形结构